TIPO ENUMERATO

Un **tipo enumerato** viene specificato tramite l'**elenco dei valori** che i dati di quel tipo possono assumere:

```
typedef enum  {a1, a2, a3, ... , an} EnumType;
```

Sono **tipi enumerabili**:
il compilatore associa a ciascun identificatore del dominio un *numero naturale*, che viene utilizzato nella valutazione di espressioni, relazioni ed assegnamenti.

→ stessa occupazione, stesso range e stessi operatori di **int**

```
/* a1, a2, a3,..., an  usabili come costanti */
EnumType var1,var2;
var1 = a3;
```

ESEMPI:

```
typedef   enum {lu, ma, me, gi, ve, sa, do}
Giorni;

typedef   enum {cuori, picche, quadri, fiori}
Carte;

Carte   C1, C2, C3, C4, C5;
Giorni  Giorno;

if (Giorno == do) /* giorno festivo */
else /* giorno feriale */
```

L'utilizzo di tipi ottenuti per enumerazione rende più leggibile il codice.

NOTE sui TIPI ENUMERATI

Un identificatore di un valore scalare definito dall'utente deve comparire nella definizione di **un solo** tipo enumerato.

```
typedef   enum {lu, ma, me, gi, ve, sa, do}
Giorni;

typedef   enum {lu, ma, me} PrimiGiorni;
```
scorretto

Un tipo enumerato è **totalmente ordinato** (*Integral Type*).

Su un dato di tipo enumerato sono applicabili tutti gli **operatori relazionali**:

lu < ma ❯ vero
lu >= sa ❯ falso
cuori < quadri ❯ vero

E' anche possibile specificare un valore naturale a cui associare i singoli simboli:

```
typedef   enum {gen, feb, mar, ...} Mesi;
```
gen → 0, feb → 1, etc

```
typedef   enum {gen=1, feb, mar, ...} Mesi;
```
gen → 1, feb → 2, etc

```
typedef   enum {gen=1, feb=4, mar, ...} Mesi;
```
gen → 1, feb → 4, mar → 5, etc

ATTENZIONE:

Non c'è controllo sugli estremi dell'intervallo di interi utilizzato!

```
m1 = 15;
```
viene accettato ed eseguito!

IL TIPO BOOLEAN

È un tipo predefinito in altri linguaggi di programmazione

Non è previsto in C, dove però:

- il valore 0 (zero) indica **FALSO**
- ogni valore diverso da 0 indica **VERO**

Vengono considerati falsi:

0 '\0' 0.0 5-5 EOF

Vengono invece considerati veri:

5 'A' 2.353*2

Per definirlo anche in C:

```
typedef enum {false, true} Boolean;
Boolean      flag1,flag2;

   flag1 = true;
   if (flag1) ...      /* flag vale 1 */

   flag2 = -37         /*  !!!  */
   if(flag2) ...    /* funziona lo stesso! */
```

EQUIVALENZA TRA TIPI DI DATO

Quando due oggetti hanno lo stesso tipo?

Equivalenza strutturale:
se si distingue tra i tipi in base alla loro struttura.

```
typedef   int    MioIntero;
typedef   int    NuovoIntero;
MioIntero    A;
NuovoIntero  B;
```

In questo caso, A e B hanno *lo stesso tipo*.

Equivalenza nominale:
due dati sono di tipo equivalente solo se compaiono nella stessa dichiarazione, oppure
se l'identificatore di tipo dell'uno è definito uguale all'identificatore di tipo dell'altro.

```
typedef   int    MioIntero;
typedef   int    NuovoIntero;
MioIntero    A;
NuovoIntero  B;
```

In questo caso, A e B vengono considerati *di tipo diverso*.

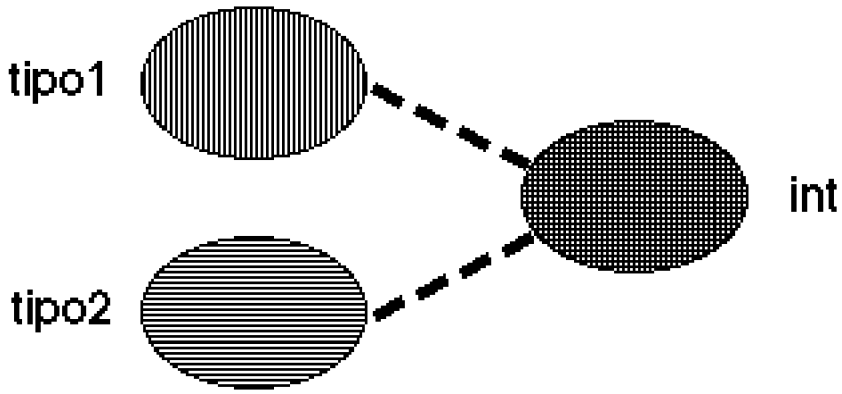
Può dipendere dalla implementazione del linguaggio.

In genere, in C vale l'equivalenza strutturale.

EQUIVALENZA DI TIPO IN C

Vale l'equivalenza strutturale

```
typedef int  tipo1;
typedef int  tipo2;
```



Si possono quindi usare insieme variabili dei due tipi:

```
tipo1     v1;
tipo2     v2;

... v1 + v2 ...
```

UNICITÀ E ORDINE DI DEFINIZIONE DEGLI IDENTIFICATORI

Dichiarazioni di costanti, tipo, variabile e tipi enumerati associano a un **identificatore** una costante, un tipo o una variabile.

L'associazione dell'identificatore con l'oggetto che denota deve essere **unica** per tutto il suo ***campo di azione.***

ESEMPIO

```
typedef   char   Intervallo;
Intervallo  X, Y;
double      Y, Z, Intervallo;
```

E' scorretta perché i simboli **`Y`** e **`Intervallo`** sono definiti due volte.

Regola:
La dichiarazione di un identificatore deve *precederne l'uso*.

```
const  int  Xmin=-Xmax;
            Xmax=100;        /*  Scorretta!   */
```